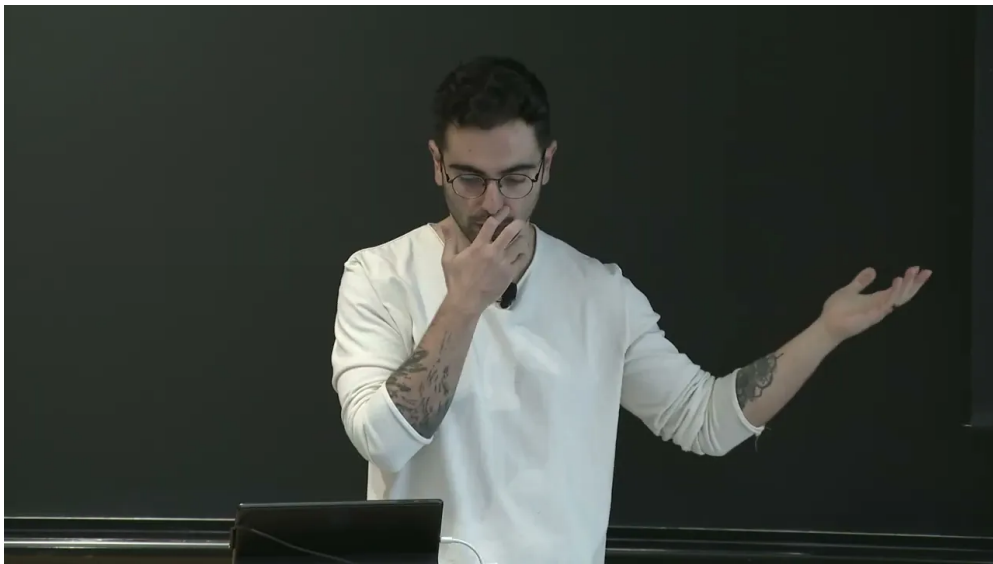


课程笔记

MIT RES.9-009 第 04 讲：Julia 中的微分方程与绘图

MIT RES.9-009 课程整理 & Codex

2026 年 4 月 15 日



视频作者/频道：MIT Video Productions

发布日期：2025-04-10（讲课日期：2025-01-14）

视频时长：12:57

视频链接：<https://www.youtube.com/watch?v=6EaolLVhnug>

目录

1	课程定位与 workflow	2
1.1	本章小结	2
2	ModelingToolkit: 从符号系统到 Lotka-Volterra	2
2.1	本章小结	4
3	Izhikevich: 手工尖峰、参数改写与外部电流	4
3.1	本章小结	5
4	动态电流与结构化化简	5
4.1	本章小结	7
5	Julia 生态协作与不确定性传播	7
5.1	本章小结	7
6	Makie: 后端、布局与组合绘图	7
6.1	本章小结	9
7	Spike plotting: 把仿真结果变成统计图	9
7.1	本章小结	10
8	总结与延伸	10
8.1	本章小结	11

证据范围

本讲没有找到官方 slide PDF；正文依赖两条主证据链：课程页 HTML/结构化页面，和嵌入视频的字幕与元数据。为了避免把缺失资源当成教学内容，相关缺口已记录到 `omission_log.jsonl`，正文只展开可核验的课程材料。

1 课程定位与 workflow

这一讲把两件事放在一起讲：**微分方程的符号建模和结果的布局式绘图**。前半部分来自 `intro_diffEq.ipynb`，后半部分来自 `intro_plot.ipynb`；它们分别对应 ModelingToolkit 与 Makie，正好构成 Neuroblox 里最基础也最常用的两条工作线。

讲者的意思很直接：如果你要在 Julia 里做计算神经科学，先要会用符号方式写系统，再要会把系统的结果组织成可读的图。前者解决“模型怎么写”，后者解决“模型怎么讲给别人看”。这两步连起来，才是一个完整的 research workflow。

官方课程页没有把所有内容做成单独 slide PDF，但页面本身已经给出了 notebook 入口、文字说明、代码块和图片。因此本讲的结构不按“屏幕截图顺序”展开，而按“建模 -> 求解 -> 改参数 -> 动态输入 -> 绘图 -> 统计”这条主线整理。

这讲的核心接口

ModelingToolkit 负责把 ODE/DAE 写成符号系统，`OrdinaryDiffEq` 负责求解，Makie 负责把轨迹、布局和 spike 统计画出来。只要掌握这三层接口，就已经能完成本讲的 notebook。

1.1 本章小结

本讲不是在讲两个无关主题，而是在讲同一个工作流的上下两端：先用符号建模和数值求解得到时间序列，再用 Makie 把时间序列转成结构清楚的图。

2 ModelingToolkit：从符号系统到 Lotka-Volterra

ModelingToolkit 的关键价值，在于它允许你用符号方式定义方程系统，而不是先手写一堆索引再把它们塞进黑箱求解器。课程页面先拿 Lotka-Volterra 作为入门例子，目的不是生态学本身，而是让你先建立“两个变量互相影响、系统随时间演化”的直觉。

$$\dot{x} = \alpha x - \beta xy, \quad \dot{y} = \delta xy - \gamma y \quad (1)$$

这类写法对应的不是静态关系，而是状态随时间的更新规则。真正重要的是，变量名、参数名和微分算子都保留在符号层；这样一来，后续求解、替换参数、提取变量和绘图都能直接围绕这些符号展开。

```
1 using ModelingToolkit
2 using ModelingToolkit: t_nounits as t, D_nounits as D
3 using OrdinaryDiffEq
4
5 @variables x(t) y(t)
6 @parameters a b g d
```

```

7 eqs = [D(x) ~ a*x - b*x*y,
8       D(y) ~ d*x*y - g*y]
9
10 sys = ODESystem(eqs, t, [x, y], [a, b, g, d])
11 prob = ODEProblem(structural_simplify(sys), [x => 5.0, y => 2.0], (0.0, 10.0),
12                  [a => 1.5, b => 1.0, g => 3.0, d => 1.0])
13 sol = solve(prob)
14 plot(sol)

```

Listing 1: ModelingToolkit 的最小 workflow

讲者还特别点出，求解后你可以像访问字典一样访问具体变量，例如 `sol[x]`。这很重要，因为它说明求解器返回的不是一团匿名数组，而是保留符号语义的 `solution object`。

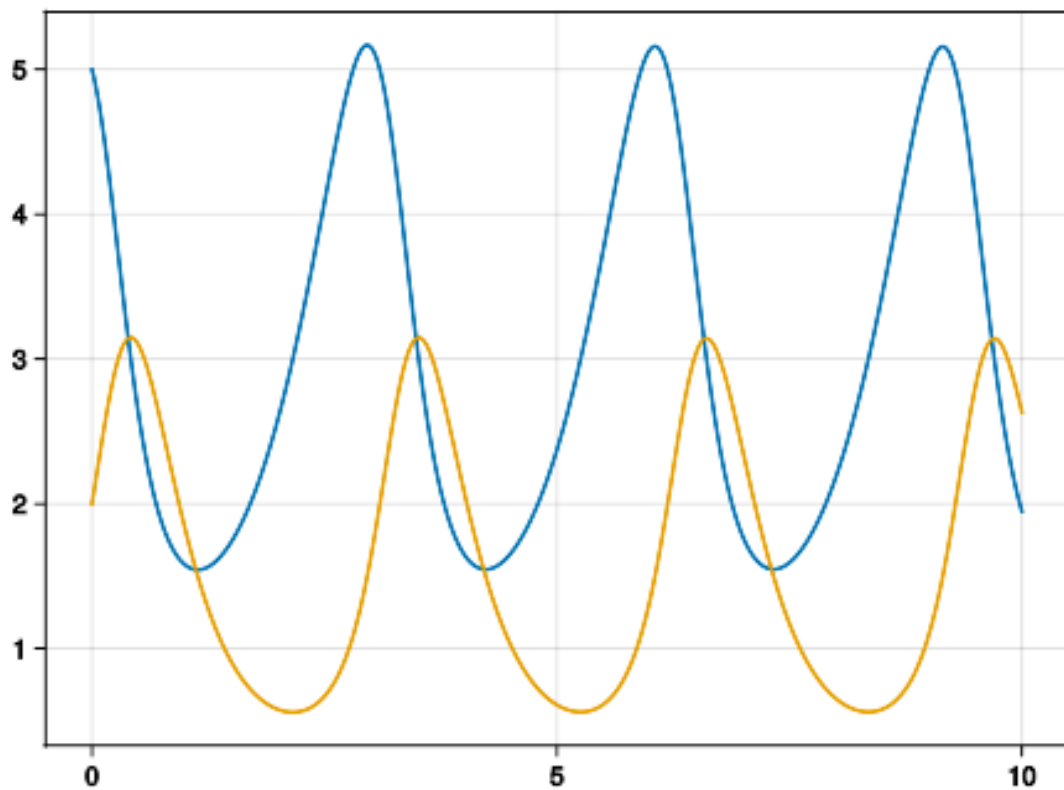


图 1: Lotka-Volterra 的时间轨迹示意。课程用这个例子先建立“符号方程 -> 求解 -> 变量访问 -> 作图”的基本链条。

讲者还借这个阶段强调了 ModelingToolkit 的模块化风格。一个系统可以先作为独立模块定义出来，再通过 `port` 接口连到另一个模块，最后再组装成更大的系统。视频里用 `mass` 和 `damper` 的类比说明：单个部件可以先写好变量和方程，再把接口暴露出来，最后把两个模型通过连接端口拼成一个新的系统。

模块化建模的意义

这一层不是为了“多写几层抽象”，而是为了让一个复杂系统可以从单元模型逐步组合出来。课程特别强调 `model macro`、嵌套模型和端口连接，就是在说明 Neuroblox 也沿着同样的设计哲学工作。

课程随后把视角扩展到 SCIMLE 生态: ModelingToolkit 只是其中一部分, PDE、UQ、Universal Differential Equations 等工具都在同一生态里协同工作。这里的关键结论是, Julia 里不同包虽然可以由不同团队独立维护, 但仍然能在同一个建模流程里互操作。

2.1 本章小结

ModelingToolkit 的重点不是“能不能写 ODE”，而是“能不能用符号保留系统结构”。一旦结构保住了，求解、替换参数和后续分析就都顺手了。

3 Izhikevich: 手工尖峰、参数改写与外部电流

课程接着把同样的建模方式推进到 Izhikevich neuron。这里的重点不再是两变量动力学本身，而是尖峰事件要如何被“手工”表达出来：当电压越过阈值时，系统要执行 `reset`，把电压拉回更极化的值，同时可能还要更新其他变量。

$$D(v) \sim 0.04v^2 + 5v + 140 - u + I, \quad D(u) \sim a(bv - u)$$

尖峰事件可以写成阈值触发加重置规则：

$$v > 30.0 \Rightarrow [u \sim u + d, v \sim c]$$

这段内容的教学重点有两个。第一，尖峰并不是连续 ODE 自动“长出来”的，需要用离散事件显式描述。第二，不同参数会让同一个神经元进入不同 spiking regime，因此参数本身就是模型的一部分。

```
1 @variables v(t)=-65 u(t)=-13
2 @parameters a=0.02 b=0.2 c=-50 d=2 I=10
3 event = (v > 30.0) => [u ~ u + d, v ~ c]
4
5 izh_system = ODESystem(eqs, t, [v, u], [a, b, c, d, I]; discrete_events = event)
6 izh_simple = structural_simplify(izh_system)
7
8 izh_prob2 = remake(izh_prob; p = [a => 0.1, b => 0.2, c => -65, d => 2])
```

Listing 2: 阈值事件与参数回填的思路

讲者随后演示了 `remake` 的意义：如果 `ODEProblem` 已经定义好，只改参数或初始条件，就能很方便地重跑模拟。视频里强调的是从 `chattering` 切换到 `fast spiking`，这说明参数组合的变化会直接改变放电模式。

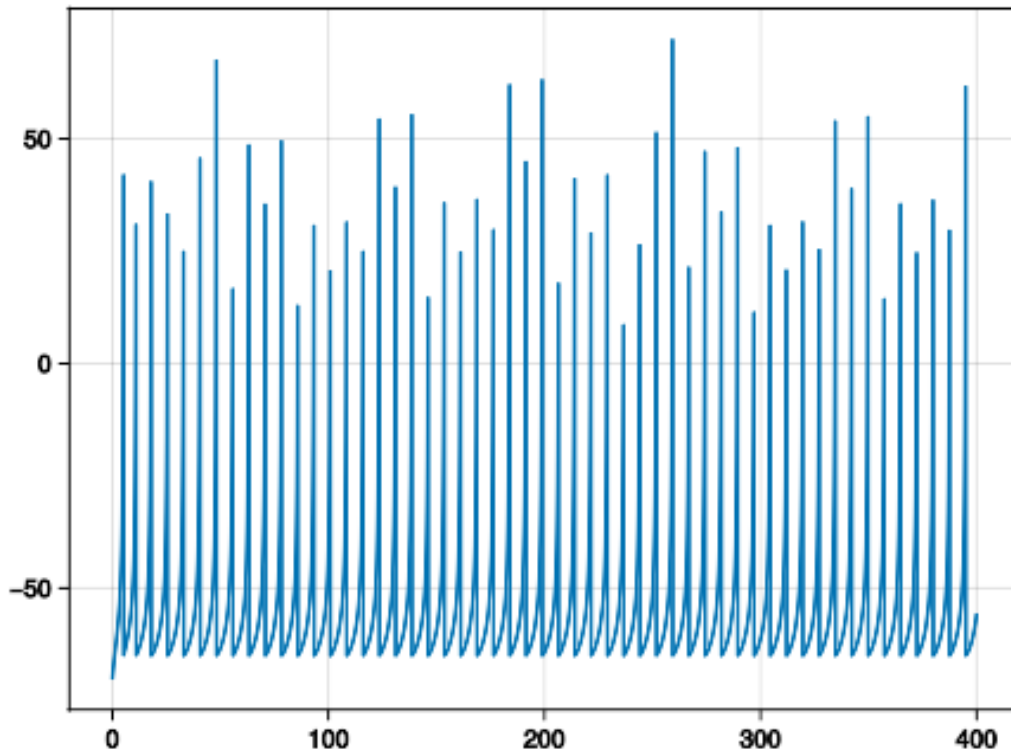


图 2: 改参数后的 Izhikevich 放电轨迹。这里能直观看到参数变化如何把尖峰模式从一种 regime 推到另一种。

3.1 本章小结

尖峰模型的核心不只是 ODE 系统本身，而是“连续动力学 + 离散事件 + 参数重写”三者一起工作。ModelingToolkit 的优势就在于把这三层都放在同一套符号语义里。

4 动态电流与结构化化简

下一步，课程把外部电流从常量参数改成了时间相关变量 $I(t)$ 。这一步很小，但非常关键，因为它把“输入”从静态常数变成了系统状态的一部分，也就更接近真实脑内输入的变化方式。

更重要的是，`structural_simplify` 会把原来含有代数方程的系统化成更容易积分的形式。视频里特意强调：原始系统里电流方程还在，但在化简后的系统里，它会被代入主方程，从而把一个 DAE 变成更容易求解的 ODE。

$$\text{DAE} \xrightarrow{\text{structural_simplify}} \text{ODE}$$

这不是纯粹的符号炫技，而是数值效率问题。ODE 的积分通常比 DAE 更直接、更稳定，也更适合后续的参数扫描与可视化。

讲者顺手也补了一个很实用的判断标准：不同系统适合不同求解器。视频里用一个相对简单的两维系统和一个更高维、更 stiff 的系统作对比，说明求解器性能、刚性和容差设置会直接改变运行表现。课程页给出的 solver guide 就是为了把这种“在什么场景用什么 solver”的判断具体化。

```
1 @variables v(t)=-65 u(t)=-13 I(t)
2 @parameters a=0.02 b=0.2 c=-65 d=8
```

```

3 eqs = [D(v) ~ 0.04*v^2 + 5*v + 140 - u + I,
4       D(u) ~ a*(b*v - u),
5       I ~ 10sin(0.5t)]
6
7 izh_system = ODESystem(eqs, t, [v, u, I], [a, b, c, d])
8 izh_simple = structural_simplify(izh_system)
9 tspan = (0.0, 400.0)
10 izh_prob = ODEProblem(izh_simple, [], tspan)
11 izh_sol = solve(izh_prob)
12 plot(izh_sol; idxs=[v, I])

```

Listing 3: 动态电流进入状态方程

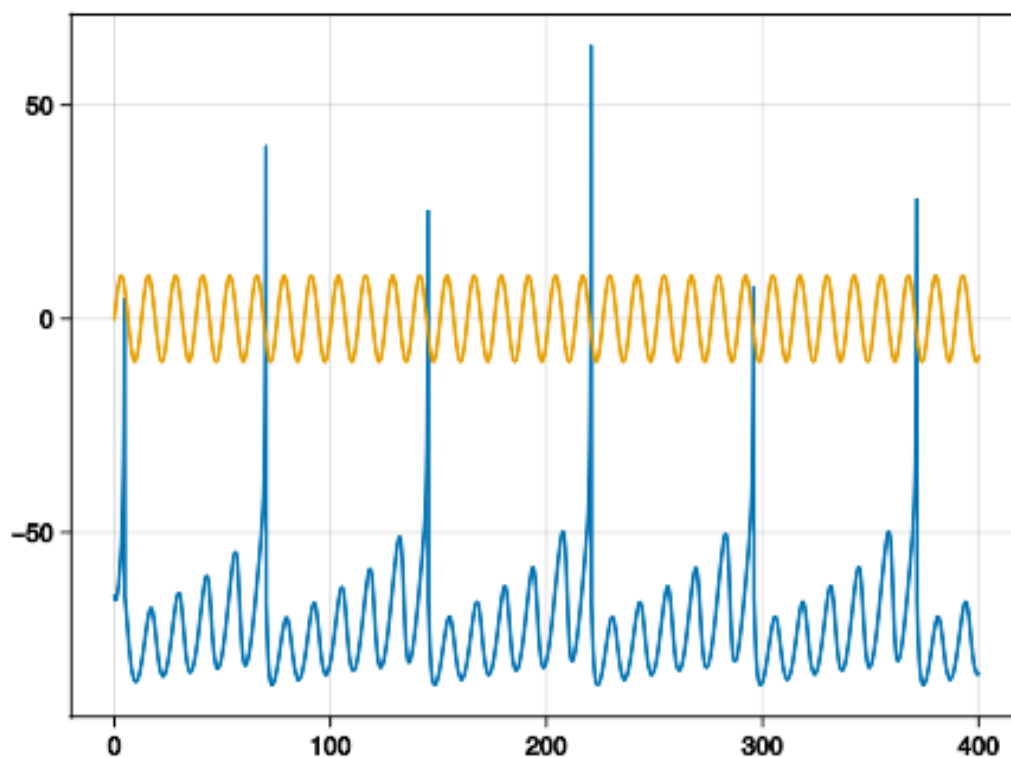


图 3: 动态外部电流进入系统后, 电压与输入一起被画出来。这里对应的是“把 I 从参数改成变量”这一步。

讲者还提到, 参数值和初始条件上的 $\pm$ 变化其实可以来自测量不确定性; 把这些不确定性显式带入模型, 得到的就不再是一条线, 而是传播后的边界或带状区域。这一点在 Julia 生态里依赖的是 `Measurements` 一类工具, 后面会接着讲。

官方课程页在这一段后面给出了一个明确的练习: 把 `generalized integrate-and-fire neuron` 写成 `ODESystem` 并模拟, 然后选一个你熟悉的神经元模型做 `sensitivity analysis`, 最后把参数变化对 `spike pattern` 的影响用一张或几张图总结出来。这个练习和前面的主线完全一致, 因为它要求你把“方程定义、参数改写、数值求解和可视化”连成闭环。

4.1 本章小结

把外部电流从常量变成变量之后，模型就从“固定输入的神经元”变成了“带动态驱动的神经元”。`structural_simplify` 的作用则是让这个系统在数值上更可解。

5 Julia 生态协作与不确定性传播

课程中间有一段很值得单独拎出来：讲者不是只在讲 `ModelingToolkit` 本身，而是在讲 Julia 生态为什么能让不同包协作。`ModelingToolkit` 和 `Measurements` 并不是一开始就为彼此设计的，但 Julia 的组合式设计让它们可以无缝协同。

这段话的含义很具体。参数值和初始条件里的 $\pm$ 量，不只是“误差条”的装饰，而是测量不确定性的编码；一旦这些量进入方程，求解器就会把它们一并传播到最终轨迹里。于是你得到的不是单条曲线，而是带边界的解。

不确定性不是附加项

在这个工作流里，不确定性不是解算完再贴上去的阴影，而是直接作为输入的一部分进入系统。这样得到的 solution 才能反映“测量误差如何穿过动力学被放大或压缩”。

讲者顺手也把这一点和 Julia 的包生态联系起来：不同包可以分别由不同团队开发，但由于语言层面的可组合性，它们仍能协作。这也是 `Neuroblox` 依赖 Julia 的一个实际原因。

课程前面提到的性能对比也在这里得到回扣：讲者明确说，`SCIMLE` 的 solver 工具链文档非常完整，能覆盖不同 stiffness 和 tolerance 需求；同一颜色编码在不同语言示例里保持一致，说明这个生态的目标不是“换一种语法而已”，而是让你把已有模型平滑迁移到 Julia 并复核结果。

课程页的参考文献也很直接：一条是 `Izhikevich 2003` 的经典尖峰模型论文，另一条是 `ModelingToolkit` 的 composable graph transformation 论文和官方文档。这说明本讲并不是临时拼出来的教程，而是直接对齐了 `SciML` 的方法论文和文档链条。

这段的核心结论

`ModelingToolkit` 管系统，`Measurements` 管不确定性，而 Julia 让两者能在同一条方程链上工作。结果是你不仅能模拟，还能把测量误差一起送进模型。

5.1 本章小结

这部分实际上讲的是 Julia 生态为什么适合科学建模：不是单个包“很强”，而是不同包之间能自然拼接，因此模型、输入和不确定性可以一起进入同一个求解流程。

6 Makie：后端、布局与组合绘图

进入绘图部分后，课程先把 `Makie` 讲成一个生态，而不是单个函数库。`Makie` 负责统一绘图接口，真正的渲染则由后端完成。课程页点名了四个后端，讲者则明确说在这门课里会用 `CairoMakie`，因为它适合 2D 静态图，而且不需要 GPU。

表 1: Makie 讲到的几个后端

后端	典型场景	课程里的定位
CairoMakie	2D 静态图、出版质量输出	本讲默认选择
GLMakie	GPU 加速的交互式 2D/3D 图	适合重交互或重计算场景
WGLMakie	Web 交互式图	适合浏览器展示
RPRMakie	光线追踪 / 更拟真渲染	仍偏实验性

Makie 的一个优点是布局很自然。讲者反复强调, Figure 可以当作一个二维网格来用, `fig[1,1]`、`fig[2,1]` 之类的位置能直接嵌入各自的 Axis。这样就能在一个窗口里同时放多张图, 而不是分散成很多窗口。

```

1 using CairoMakie
2
3 fig = Figure(size = (700, 500))
4 ax1 = Axis(fig[1, 1], title = "Line_plot")
5 ax2 = Axis(fig[2, 1], title = "Scatter_plot")
6 lines!(ax1, seconds, measurements)
7 scatter!(ax2, seconds, measurements)
8 hidexdecorations!(ax1, grid = false)
9 fig

```

Listing 4: 一个最常见的 Makie 布局骨架

课程还特别提到, Makie 的布局和普通 `lines(seconds, measurements)` 这种简写相比, 多了对坐标轴和排版控制。若要做正式汇报或论文图, 这种控制就非常值钱。

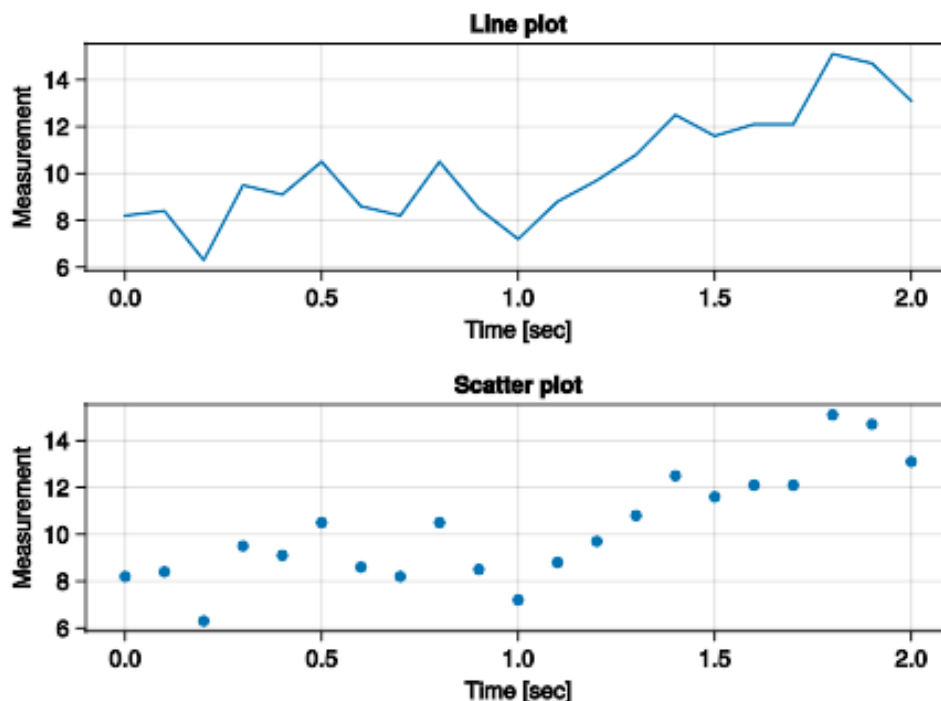


图 4: Makie 的两面板布局示意。课程用它说明 Figure 和 Axis 可以像网格一样组合。

讲者还补充说，这种布局能力正是新用户容易上手的地方。一个 panel 里还可以再嵌套多个 subplot，形成更复杂的分块结构；课程里提到的 panel A、legend、brain image 和多个子图，都是同一套布局系统的自然结果。若你来自 R 的 ggplot2，可以把 AlgebraOfGraphics 看成类似 grammar of graphics 的入口；若你来自 Python，base Makie 的调用风格会更像 matplotlib，只是排版和组合更统一。

Makie 的 extended ecosystem 也不是摆设。课程明确提到 neuroscience 里的 power spectrum、scalp map、source localization、event-related potentials 和 pair plots，甚至 multibody 与 robotic arms 这类例子。换言之，Makie 不是只能画一般曲线，它更像一套可以跨领域复用的视觉语法。

6.1 本章小结

Makie 不是单纯的“画图包”，而是一个统一接口加多后端的绘图生态。对 Neuroblox 来说，这意味着仿真结果、布局和发表级输出可以在同一套语义下完成。

7 Spike plotting: 把仿真结果变成统计图

最后一段把前面的两条主线合到一起：先仿真，再画更有信息量的图。课程选的是 fast-spiking Izhikevich 神经元，并在特定时刻改变外部电流，然后按窗口统计 spike 数。

这个例子很适合说明 Makie 的实际价值。你不只是把电压轨迹画成一条线，而是把 spike 计数、刺激时刻和时间窗都组织成一张图。这样一来，观众不仅看到“有尖峰”，还能看到“刺激前后尖峰统计如何变化”。

```

1 spikes = zeros{Int, length(t_range) - 1}
2 for i in eachindex(t_range[1:end-1])
3     idxs = findall(t -> t_range[i] <= t <= t_range[i + 1], timepoints)

```

```

4     spikes[i] = count(V_izh[idxs] .> spike_threshold)
5 end

```

Listing 5: 按时间窗统计 spike 的核心思路

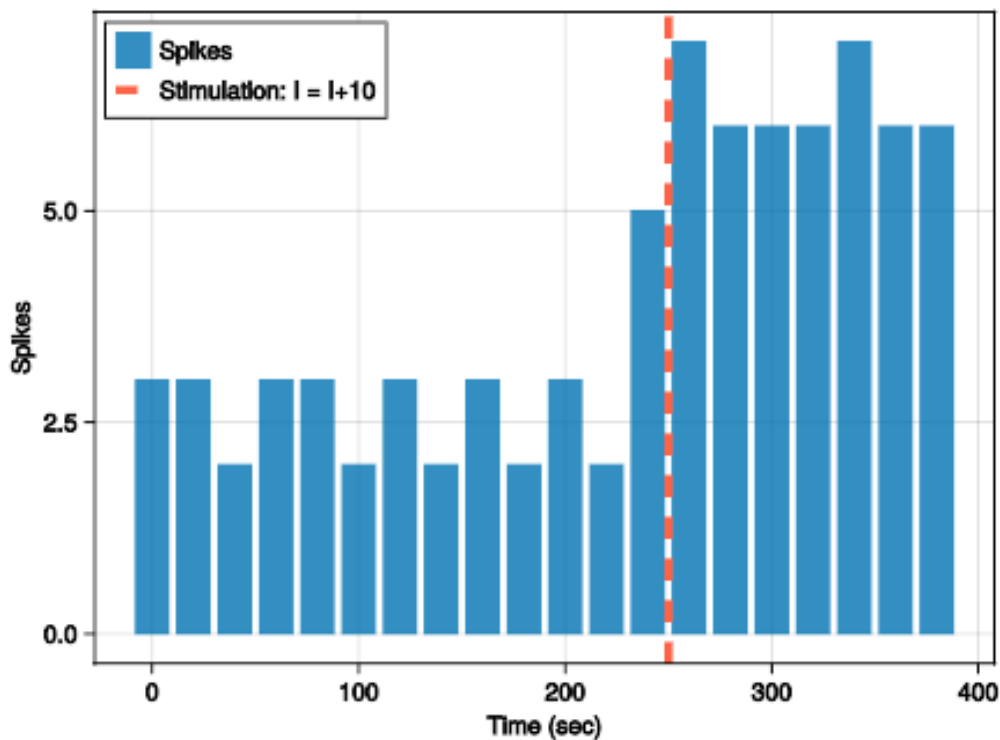


图 5: spike 统计图：红色虚线标出刺激开始时刻，柱状图则显示不同时间窗内的 spike 数。

讲者还顺带提醒，Makie 的这个例子可以直接从课程网站上的 notebook 复现：同样的材料同时存在于网页和 notebook 里，所以最稳妥的做法就是先照着 `intro_diffeq.ipynb` 做，再切到 `intro_plot.ipynb`。

网页上的 plotting challenge 也把你重新指回前一部分：如果你要继续练习，就先回到 Differential Equations in Julia 的 notebook，把前面定义的神元系统补完整，再回来把结果换一种图法呈现。

7.1 本章小结

这一段把“模拟”和“展示”真正接上了：不是只看单条轨迹，而是把事件计数、刺激窗口和时间轴放进同一张图里，让结果的结构一眼可见。

8 总结与延伸

这讲的主线很统一：用 ModelingToolkit 写动力系统，用 OrdinaryDiffEq 求解，用 Makie 展示结果。Lotka-Volterra 用来建立符号 ODE 的直觉，Izhikevich 用来说明尖峰事件和参数重写，动态电流和 `structural_simplify` 用来展示 DAE 到 ODE 的化简，Makie 则把布局和 spike 统计变成清晰的视觉结果。

课后顺序

先做 `intro_diff_eq.ipynb`，再做 `intro_plot.ipynb`。课程页已经把同样的材料放在网页和 notebook 两边，最好的学习方式就是按这个顺序把整套流程跑一遍。

官方课程页给了两类参考：一类是 Izhikevich neuron 的经典论文，另一类是 ModelingToolkit 与 Makie 的文档。就这讲而言，最重要的不是死记某个函数名，而是把“符号建模 -> 求解 -> 参数修改 -> 动态输入 -> 布局绘图”这个完整链条记住。

讲者在结尾也明确说了，Notebook 和 website 的内容是对齐的，所以如果你在课堂上没跟上，最稳妥的补课顺序就是先把网页和 notebook 各跑一遍，再回头看这讲的几个关键图。

8.1 本章小结

如果只记一句话，这讲要传达的是：Neuroblox 的基础能力不是“会不会画图”，而是“能不能把动力学模型写对、解对、改对、画对”。这四件事在 Julia 里正好可以连成一条流水线。